Home: http://www.postgresql.org/

SQL: https://www.postgresql.org/docs/current/sql-commands.html
[Reference] SQL commands: https://www.postgresql.org/docs/current/sql-commands.html

*** Defaults**
default host: localhost
default port: 5432
default user: postgres
default password: -
default schema: public

*** Data types**
Pseudo data types: https://www.postgresql.org/docs/current/datatype-pseudo.html
Data types: https://www.postgresql.org/docs/current/datatype.html

```
// Numeric
SMALLINT
INTEGER
BIGINT
DECIMAL
NUMERIC
REAL
DOUBLE PRECISION
SMALLSERIAL               small autoincrementing
SERIAL                    medium
BIGSERIAL                 large

// Text
BPCHAR(10)       blank padded
BPCAHR           unlimited length
TEXT             unlimited length

... etc ...


// Pseudo
void                 for FUNCTION RETURNS void
unknown              NULL
record               any table row type

any
anyelement
anyarray
anynonarray
anyenum
...etc...
```

*** [Non-Standard] Infinity, NaN**
IEE754: https://en.wikipedia.org/wiki/IEEE_754

NaN, Infinity
NOT in SQL standard, but standard IEE754 numeric value
Only supported for **arbitrary precision** types i.e. NUMERIC

```
'NaN'
NaN                   -- ERROR. Must be quoted
'Infinity'
'-Infinity'
```

```
SELECT 'Infinity';                    -- ERROR. unspecified type

SELECT 'Infinity'::NUMERIC;           -- OK

SELECT 'Infinity'::REAL;              -- OK

SELECT 'Infinity'::INT;               -- ERROR. Not supported.

SELECT 'Infinity'::NUMERIC(10);       -- ERROR. Overflow

SELECT 'Infinity'::NUMERIC + 1;
```

```
-- NaN arithmetic
'NaN'::numeric + 1                -- NaN
'NaN'::numeric - 1                -- NaN

--
'Infinity'::numeric + 1                         -- infinity
'Infinity'::numeric - 1                         -- infinity
'Infinity'::numeric + 'Infinity'::numeric       -- infinity
'Infinity'::numeric + '-Infinity'::numeric      -- NaN
'Infinity'::numeric - 'Infinity'::numeric       -- NaN
```

*** [Non-standard] time special values**
Ref: https://www.postgresql.org/docs/current/datatype-datetime.html

e.g.

```
-- Special values
SELECT TIMESTAMP WITH TIME ZONE 'now';     -- e.g. 2025-01-13
10:20:30.12345+09
SELECT TIMESTAMP 'now';                    -- e.g. 2025-01-13
10:20:30.12345
SELECT DATE 'now';                         -- 2025-01-13
SELECT TIME 'now';                         -- 10:20:30.12345
SELECT TIME WITH TIME ZONE 'now';          -- 10:20:30.12345+09

-- functions
SELECT CURRENT_TIMESTAMP;
```

```
'epoch'
'infinity'
'-infinity'
'now'
'today'
'yesterday'
'allballs'
```

*** User defined type**
CREATE TYPE: https://www.postgresql.org/docs/current/sql-createtype.html

```
CREATE TYPE name AS
    ( [ attribute_name data_type [ COLLATE collation ] [, ... ] ] )

CREATE TYPE name AS ENUM
    ( [ 'label' [, ... ] ] )

CREATE TYPE name AS RANGE (
    SUBTYPE = subtype
    [ , SUBTYPE_OPCLASS = subtype_operator_class ]
    [ , COLLATION = collation ]
    [ , CANONICAL = canonical_function ]
    [ , SUBTYPE_DIFF = subtype_diff_function ]
    [ , MULTIRANGE_TYPE_NAME = multirange_type_name ]
)

CREATE TYPE name (
    INPUT = input_function,
    OUTPUT = output_function
    [ , RECEIVE = receive_function ]
    [ , SEND = send_function ]
    [ , TYPMOD_IN = type_modifier_input_function ]
    [ , TYPMOD_OUT = type_modifier_output_function ]
    [ , ANALYZE = analyze_function ]
    [ , SUBSCRIPT = subscript_function ]
    [ , INTERNALLENGTH = { internallength | VARIABLE } ]
    [ , PASSEDBYVALUE ]
    [ , ALIGNMENT = alignment ]
    [ , STORAGE = storage ]
    [ , LIKE = like_type ]
    [ , CATEGORY = category ]
    [ , PREFERRED = preferred ]
    [ , DEFAULT = default ]
    [ , ELEMENT = element ]
    [ , DELIMITER = delimiter ]
    [ , COLLATABLE = collatable ]
)
```

```
CREATE TYPE name
```

CREATE TYPE
Supports creating custom type.
Only **composite type** is standard. The rest are PG extensions

1) [SQL] **Composite** type aka **Row** type
Ref: https://www.postgresql.org/docs/current/rowtypes.html
Standard SQL
Equivalent to a **table row**.
e.g.

```
CREATE mypoint AS (x INT, y INT);

SELECT (1,2)::mypoint;

SELECT ((1,2)::mypoint).x;
```

2) [PG] **ENUM** type
Ref: https://www.postgresql.org/docs/current/datatype-enum.html
e.g.

```
CREATE mylabel AS ENUM ('a', 'b');

SELECT 'a'::mylabel;
```

3) [PG] **RANGE** type
Ref: https://www.postgresql.org/docs/current/rangetypes.html
Custom range type
e.g.

```
CREATE myrange AS RANGE (SUBTYPE INT);

SELECT myrange(0, 10);                  -- [0,10)
SELECT myrange(0, 10, '[]');            -- [0,10]

SELECT mymultirange(                    -- {[0,10),[10,20)}
  myrange(0,10),myrange(10,20)
);
```

4) [PG] **Base** type
A **new scalar** type

*** PREPARE**
PREPARE: https://www.postgresql.org/docs/current/sql-prepare.html
DEALLOCATE: https://www.postgresql.org/docs/current/sql-deallocate.html
EXECUTE: https://www.postgresql.org/docs/current/sql-execute.html

```
PREPARE name [ ( data_type [, ...] ) ] AS statement         --
data_type is optional

EXECUTE name(parameter_values);

EXPLAIN EXECUTE name(parameter_values);

DEALLOCATE [ PREPARE ] { name | ALL }

EXECUTE name [ ( parameter [, ...] ) ]
```

PREPARE
Prepared statemtnt
A SQL statement with **parameters** i.e. a statements **template**.
A prepared statement is accessible only in the **current session**.
In particular,

PREPARE & Optimization
PREPARE is also a form of optimization.
PREPARE allows computing a **generic query plan** beforehand, and **reusing** the query plan on each EXECUTE.
However, it may also compute a **custom query plan** per EXECUTE.

e.g. basic usage

```
-- Step 1: Define prepared statemtent
PREPARE s AS
  SELECT * FROM mytable WHERE value >= $1 AND type = $2;

PREPARE s (int, text) AS
    -- same with explicit argument types
  SELECT  * FROM mytable WHERE value >= $1 AND type = $2;
    -- optional


-- Step 2: Execute
EXECUTE s(1, 'a');


-- Step 3: EXPLAIN
EXPLAIN EXECUTE s(1, 'a');


-- Step 4: remove
DEALLOCATE s;
DEALLOCATE ALL;
```

e.g. **pg_prepared_statements** to see defined prepared statements

```
> select * from pg_prepared_statements \gx

-[ RECORD
1 ]---+-----------------------------------------------------------------
name            | f
statement       | prepare f as select * from t1 where id >= $1 and
id <= $2;
prepare_time    | 2025-04-08 05:45:48.863509+09
parameter_types | {bigint,bigint}
result_types    | {bigint,integer,numeric}
from_sql        | t
generic_plans   | 0
custom_plans    | 0
...
```

```
-- Basic
PREPARE s AS
PREPARE s (int, text) AS              -- optional data type
  SELECT *
  FROM mytable
  WHERE value >= $1 AND type = $2;
-- Execute
EXECUTE s(1, 'a');                -- execute with arguments


-- case: no arguments
PREPARE hello AS
  SELECT 'hello';
EXECUTE hello;                    -- OK
EXECUTE hello();                  -- ERROR



-- GOTCHA: arguments can only be values
PREPARE s AS                  -- ERROR: "syntax error at or near
$1"
  SELECT * FROM $1;           -- can't use as table identifier
PREPARE s AS                  -- ERROR: "syntax error at or near
$1"
  SELECT $1, $2 FROM a;       -- can't use as column identifier
```

==========

== FUNCTION/PROCEDURE/ROUTINE ==

User defined functions: https://www.postgresql.org/docs/current/xfunc.html

SQL functions: https://www.postgresql.org/docs/current/xfunc-sql.html

https://www.postgresql.org/docs/current/sql-createfunction.html

```
CREATE [ OR REPLACE ] FUNCTION
    name ( [ [ argmode ] [ argname ] argtype [ { DEFAULT | = }
default_expr ] [, ...] ] )
    [ RETURNS rettype
      | RETURNS TABLE ( column_name column_type [, ...] ) ]
  { LANGUAGE lang_name
    | TRANSFORM { FOR TYPE type_name } [, ... ]
    | WINDOW
    | { IMMUTABLE | STABLE | VOLATILE }
    | [ NOT ] LEAKPROOF
    | { CALLED ON NULL INPUT | RETURNS NULL ON NULL INPUT |
STRICT }
    | { [ EXTERNAL ] SECURITY INVOKER | [ EXTERNAL ] SECURITY
DEFINER }
    | PARALLEL { UNSAFE | RESTRICTED | SAFE }
    | COST execution_cost
    | ROWS result_rows
    | SUPPORT support_function
    | SET configuration_parameter { TO value | = value | FROM
CURRENT }
    | AS 'definition'
    | AS 'obj_file', 'link_symbol'
    | sql_body
  } ...
```

FUNCTION
PROCEDURE
ROUTINE
A **FUNCTION** has a **RETURN** value (but may be **VOID**).
A **PROCEDURE** has no return value.
A **ROUTINE** is a collective way to refer to functions and procedure.
i.e. functions and procedures are both routines.

*** Function volatility**
Volatility: https://www.postgresql.org/docs/current/xfunc-volatility.html

Volatility
Volatility represents the degree of **stability** of a function.
3 levels of volatility:

1) **immutable**: always the same result in any execution.
  e.g. sum
    all window/aggregate functions

2) **stable**: always the same result in the **current** execution.
  e.g. CURRENT_TIMESTAMP() since the time is fixed per transaction

3) **volatile (default)**: otherwise
  i.e. the result may **change** on each call.

e.g. set_config()

By default, all **user defined functions** i.e. CREATE FUNCTION are volatile.

e.g. check volatility of a function
Use **\df+** command or **pg_proc**

```
-- with \d command
> \df+ max*

List of functions
   Schema   | Name |      Result data type       |     Argument
data types     | Type | Volatility | Parallel |  Owner   | Security
| Access privileges | Language |  Internal name  |
Description
------------+------+-----------------------------+-----------------
------------+------+------------+----------+----------+----------+-
-----------------+----------+-----------------+-------------------
-------------------------------------
 pg_catalog | max  | anyarray                    | anyarray
| agg  | immutable  | safe     | slpadmin | invoker  | NULL
| internal | aggregate_dummy | maximum value of all anyarray input
values
 pg_catalog | max  | anyenum                     | anyenum
| agg  | immutable  | safe     | slpadmin | invoker  | NULL
| internal | aggregate_dummy | maximum value of all enum input
values
 pg_catalog | max  | bigint                      | bigint
....

-- With pg_catalog
> SELECT DISTINCT proname, provolatile
FROM pg_proc
WHERE proname = 'max';

 proname | provolatile
---------+-------------
 max     | i
```

window functions: all immutable

```
> SELECT proname, provolatile
FROM pg_proc
WHERE prokind = 'w';                    -- window function

   proname    | provolatile
--------------+-------------
 row_number   | i
 rank         | i
 dense_rank   | i
```

```
percent_rank | i
cume_dist    | i
ntile        | i
lag          | i
lag          | i
lag          | i
lead         | i
lead         | i
lead         | i
first_value  | i
last_value   | i
nth_value    | i
```

aggregate functions: all immutable EXCEPT json functions

```
> SELECT distinct proname, provolatile
FROM pg_proc
WHERE prokind = 'a'                          -- aggregate function
ORDER BY proname;

       proname        | provolatile
----------------------+-------------
 array_agg            | i
 avg                  | i
 bit_and              | i
 bit_or               | i
 bit_xor              | i
 bool_and             | i
 bool_or              | i
 corr                 | i
 count                | i
 covar_pop            | i
 covar_samp           | i
 cume_dist            | i
 dense_rank           | i
 every                | i
 json_agg             | s
 json_object_agg      | s
 jsonb_agg            | s
 jsonb_object_agg     | i
 max                  | i
 min                  | i
 mode                 | i
 percent_rank         | i
 percentile_cont      | i
 percentile_disc      | i
 range_agg            | i
 range_intersect_agg  | i
 rank                 | i
 regr_avgx            | i
```

```
 regr_avgy           | i
 regr_count          | i
 regr_intercept      | i
 regr_r2             | i
 regr_slope          | i
 regr_sxx            | i
 regr_sxy            | i
 regr_syy            | i
 stddev              | i
 stddev_pop          | i
 stddev_samp         | i
 string_agg          | i
 sum                 | i
 var_pop             | i
 var_samp            | i
 variance            | i
 xmlagg              | i
```

Function volatility vs performance
Volatility is important since **stable** and **immutable** functions are evaluated **only once**.

e.g.

```
-- e.g. in SELECT
SELECT
  to_char(CURRENT_TIMESTAMP, 'YYYY-MM') AS now,     -- evaluated
only once in whole
  ...
FROM a;


-- e.g. in filter
SELECT ...
FROM a
LEFT OUTER JOIN b ON
  upper(a.name) = upper(b.name)                     --
upper(a.name) evaluated only once
```

*** GOTCHA: function side effect error "... does not exist"**
Reference: https://www.postgresql.org/docs/current/xfunc-sql.html

Problem: function is allowed to have **side effects**.
e.g. may update a table
But the function query is **processed on definition**.
Hence, any objects (e.g. tables) that do not exist then will cause an error.
In particular, it will NOT see objects that will

e.g. cannot INSERT into a new table

```
create or replace function setup () returns void as $$
```

```
  create table a (value int primary key generated always as
identity);   -- OK
  insert into a default values;
         -- ERROR since table a does not exists yet
$$ language sql;

 ERROR:  relation "a" does not exist
LINE 3:   insert into a default values;
```

Syntax

```
CREATE FUNCTION f
CREATE OR REPLACE FUNCTION f
   (int, text)
   (x int, name text)
   (int DEFAULT 1, ...)
   (int = 1, ...)
```

```
CREATE FUNCTION f(x int, y int)
   RETURNS int AS
  'SELECT x + y'                        -- with arg name
  'SELECT f.x + f.y'                    -- with fully qualified arg
name
  'SELECT $1 + $2'                      -- OLD: with position
  LANGUAGE SQL
```

```
  -- RETURNS
  RETURNS INT
  RETURNS VOID                -- No return value


-- Set functions
--
-- case: defined table/composite type
CREATE FUNCTION f() SETOF INT AS          -- row of scalars
CREATE FUNCTION f() SETOF mytype AS       -- row of composite type
CREATE FUNCTION f() SETOF mytable AS      -- table row's type
```

```
> CREATE FUNCTION f()
RETURNS SETOF INT
'VALUES (1), (2)'
LANGUAGE SQL;

> SELECT * FROM f();
 f
———
 1
 2
```

```
> CREATE FUNCTION f()
  RETURNS SETOF mytable
  'SELECT * FROM mytable'
  LANGUAGE SQL;

> SELECT * FROM f();
> SELECT * FROM mytable;               -- same
```

```
-- case: anonymous table
CREATE FUNCTION f(out x int, out y int) SETOF record
CREATE FUNCTION f(out int, out int) SETOF record            -- OK.
Default column names i.e. column1, column2
CREATE FUNCTION f() SETOF record                            -- BAD.
Compiles but runtime error since no columns are defined
CREATE FUNCTION f() TABLE(x int, y int)                     -- same
CREATE FUNCTION f() SETOF TABLE(x int, y int)               --
ERROR. Don't need SETOF
```

```
> CREATE FUNCTION f(out x int, out y int) RETURNS SETOF record AS
'VALUES(1, 2), (3, 4)'
LANGUAGE SQL;

> CREATE FUNCTION f() RETURNS TABLE(x int, y int) AS
'VALUES(1, 2), (3, 4)'
LANGUAGE SQL;

> SELECT f();
   f
-------
 (1,2)
 (3,4)

> SELECT * FROM f() AS t;
 x | y
---+---
 1 | 2
 3 | 4


> CREATE FUNCTION f(out int, out int) RETURNS SETOF record AS
'VALUES(1, 2), (3, 4)'
LANGUAGE SQL;

> SELECT * FROM f() AS t;
 column1 | column2
---------+---------
       1 |       2
       3 |       4
```

==========

==Table==

CREATE TABLE: https://www.postgresql.org/docs/current/sql-createtable.html

```
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } | UNLOGGED ] TABLE
[ IF NOT EXISTS ] table_name ( [
  { column_name data_type [ STORAGE { PLAIN | EXTERNAL | EXTENDED |
MAIN | DEFAULT } ] [ COMPRESSION compression_method ] [ COLLATE
collation ] [ column_constraint [ ... ] ]
    | table_constraint
    | LIKE source_table [ like_option ... ] }
    [, ... ]
] )
[ INHERITS ( parent_table [, ... ] ) ]
[ PARTITION BY { RANGE | LIST | HASH } ( { column_name |
( expression ) } [ COLLATE collation ] [ opclass ] [, ... ] ) ]
[ USING method ]
[ WITH ( storage_parameter [= value] [, ... ] ) | WITHOUT OIDS ]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ TABLESPACE tablespace_name ]



-- Typed table
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } | UNLOGGED ] TABLE
[ IF NOT EXISTS ] table_name
    OF type_name [ (
  { column_name [ WITH OPTIONS ] [ column_constraint [ ... ] ]
    | table_constraint }
    [, ... ]
) ]
[ PARTITION BY { RANGE | LIST | HASH } ( { column_name |
( expression ) } [ COLLATE collation ] [ opclass ] [, ... ] ) ]
[ USING method ]
[ WITH ( storage_parameter [= value] [, ... ] ) | WITHOUT OIDS ]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ TABLESPACE tablespace_name ]



-- Partition
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } | UNLOGGED ] TABLE
[ IF NOT EXISTS ] table_name
    PARTITION OF parent_table [ (
  { column_name [ WITH OPTIONS ] [ column_constraint [ ... ] ]
    | table_constraint }
    [, ... ]
) ] { FOR VALUES partition_bound_spec | DEFAULT }
[ PARTITION BY { RANGE | LIST | HASH } ( { column_name |
( expression ) } [ COLLATE collation ] [ opclass ] [, ... ] ) ]
[ USING method ]
[ WITH ( storage_parameter [= value] [, ... ] ) | WITHOUT OIDS ]
```

```
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ TABLESPACE tablespace_name ]

and like_option is:

{ INCLUDING | EXCLUDING } {
    COMMENTS
  | COMPRESSION
  | CONSTRAINTS               -- CHECK constraint
  | DEFAULTS                  -- DEFAULT constraint
  | GENERATED                 -- GENERATED ... AS <expression>
  | IDENTITY                  -- GENERATED ... AS IDENTITY
  | INDEXES                   -- indexes, including UNIQUE, PRIMARY
KEY, EXCLUDING constraints
  | STATISTICS
  | STORAGE
  | ALL
}
```

partition_bound_spec

```
IN ( partition_bound_expr [, ...] ) |
    -- LIST partition

FROM ( { partition_bound_expr | MINVALUE | MAXVALUE } [, ...] )
    -- RANGE partition
  TO ( { partition_bound_expr | MINVALUE | MAXVALUE } [, ...] ) |

WITH ( MODULUS numeric_literal, REMAINDER numeric_literal )
    -- HASH partition
```

column constraint

```
[ CONSTRAINT constraint_name ]
{ NOT NULL |
  NULL |
  CHECK ( expression ) [ NO INHERIT ] |
  DEFAULT default_expr |
  GENERATED ALWAYS AS ( generation_expr ) STORED |
  GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY
[ ( sequence_options ) ] |
  UNIQUE [ NULLS [ NOT ] DISTINCT ] index_parameters |
  PRIMARY KEY index_parameters |
  REFERENCES reftable [ ( refcolumn ) ] [ MATCH FULL | MATCH
PARTIAL | MATCH SIMPLE ]
    [ ON DELETE referential_action ] [ ON UPDATE
referential_action ] }
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY
IMMEDIATE ]
```

exclude_element in an EXCLUDE

```
{ column_name | ( expression ) } [ COLLATE collation ] [ opclass
[ ( opclass_parameter = value [, ... ] ) ] ] [ ASC | DESC ] [ NULLS
{ FIRST | LAST } ]
```

table constraint

```
[ CONSTRAINT constraint_name ]
{ CHECK ( expression ) [ NO INHERIT ] |
  UNIQUE [ NULLS [ NOT ] DISTINCT ] ( column_name [, ... ] )
index_parameters |
  PRIMARY KEY ( column_name [, ... ] ) index_parameters |
  EXCLUDE [ USING index_method ] ( exclude_element WITH operator
[, ... ] ) index_parameters [ WHERE ( predicate ) ] |
  FOREIGN KEY ( column_name [, ... ] ) REFERENCES reftable
[ ( refcolumn [, ... ] ) ]
    [ MATCH FULL | MATCH PARTIAL | MATCH SIMPLE ] [ ON DELETE
referential_action ] [ ON UPDATE referential_action ] }
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY
IMMEDIATE ]
```

index parameters

```
[ INCLUDE ( column_name [, ... ] ) ]
[ WITH ( storage_parameter [= value] [, ... ] ) ]
[ USING INDEX TABLESPACE tablespace_name ]
```

referential_action in a FOREIGN KEY/REFERENCES

```
{ NO ACTION | RESTRICT | CASCADE | SET NULL [ ( column_name
[, ... ] ) ] | SET DEFAULT [ ( column_name [, ... ] ) ] }
```

CREATE TABLE AS: https://www.postgresql.org/docs/current/sql-createtableas.html

```
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } | UNLOGGED ] TABLE
[ IF NOT EXISTS ] table_name
    [ (column_name [, ...] ) ]
    [ USING method ]
    [ WITH ( storage_parameter [= value] [, ... ] ) | WITHOUT
OIDS ]
    [ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
    [ TABLESPACE tablespace_name ]
    AS query
    [ WITH [ NO ] DATA ]
```

ALTER TABLE: https://www.postgresql.org/docs/current/sql-altertable.html

```
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
    action [, ... ]

-- rename
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
    RENAME [ COLUMN ] column_name TO new_column_name
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
    RENAME CONSTRAINT constraint_name TO new_constraint_name
ALTER TABLE [ IF EXISTS ] name
    RENAME TO new_name

-- partition
ALTER TABLE [ IF EXISTS ] name
    ATTACH PARTITION partition_name { FOR VALUES
partition_bound_spec | DEFAULT }
ALTER TABLE [ IF EXISTS ] name
    DETACH PARTITION partition_name [ CONCURRENTLY | FINALIZE ]


-- location
ALTER TABLE [ IF EXISTS ] name
    SET SCHEMA new_schema
ALTER TABLE ALL IN TABLESPACE name [ OWNED BY role_name [, ... ] ]
    SET TABLESPACE new_tablespace [ NOWAIT ]


where action is one of:

    ADD [ COLUMN ] [ IF NOT EXISTS ] column_name data_type
[ COLLATE collation ] [ column_constraint [ ... ] ]
    DROP [ COLUMN ] [ IF EXISTS ] column_name [ RESTRICT |
CASCADE ]
    ALTER [ COLUMN ] column_name [ SET DATA ] TYPE data_type
[ COLLATE collation ] [ USING expression ]
    ALTER [ COLUMN ] column_name SET DEFAULT expression
    ALTER [ COLUMN ] column_name DROP DEFAULT
    ALTER [ COLUMN ] column_name { SET | DROP } NOT NULL
    ALTER [ COLUMN ] column_name SET EXPRESSION AS ( expression )
    ALTER [ COLUMN ] column_name DROP EXPRESSION [ IF EXISTS ]
    ALTER [ COLUMN ] column_name ADD GENERATED { ALWAYS | BY
DEFAULT } AS IDENTITY [ ( sequence_options ) ]
    ALTER [ COLUMN ] column_name { SET GENERATED { ALWAYS | BY
DEFAULT } | SET sequence_option | RESTART [ [ WITH ] restart ] }
[...]
    ALTER [ COLUMN ] column_name DROP IDENTITY [ IF EXISTS ]
    ALTER [ COLUMN ] column_name SET STATISTICS { integer |
DEFAULT }
    ALTER [ COLUMN ] column_name SET ( attribute_option = value
[, ... ] )
    ALTER [ COLUMN ] column_name RESET ( attribute_option
```

```
[, ... ] )
    ALTER [ COLUMN ] column_name SET STORAGE { PLAIN | EXTERNAL |
EXTENDED | MAIN | DEFAULT }
    ALTER [ COLUMN ] column_name SET COMPRESSION compression_method


    ADD table_constraint [ NOT VALID ]
    ADD table_constraint_using_index
    ALTER CONSTRAINT constraint_name [ DEFERRABLE | NOT
DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]
    VALIDATE CONSTRAINT constraint_name
    DROP CONSTRAINT [ IF EXISTS ]  constraint_name [ RESTRICT |
CASCADE ]


    DISABLE TRIGGER [ trigger_name | ALL | USER ]
    ENABLE TRIGGER [ trigger_name | ALL | USER ]
    ENABLE REPLICA TRIGGER trigger_name
    ENABLE ALWAYS TRIGGER trigger_name


    DISABLE RULE rewrite_rule_name
    ENABLE RULE rewrite_rule_name
    ENABLE REPLICA RULE rewrite_rule_name
    ENABLE ALWAYS RULE rewrite_rule_name


    DISABLE ROW LEVEL SECURITY
    ENABLE ROW LEVEL SECURITY
    FORCE ROW LEVEL SECURITY
    NO FORCE ROW LEVEL SECURITY


    CLUSTER ON index_name
    SET WITHOUT CLUSTER
    SET WITHOUT OIDS
    SET ACCESS METHOD { new_access_method | DEFAULT }
    SET TABLESPACE new_tablespace
    SET { LOGGED | UNLOGGED }
    SET ( storage_parameter [= value] [, ... ] )
    RESET ( storage_parameter [, ... ] )


    INHERIT parent_table
    NO INHERIT parent_table
    OF type_name
    NOT OF
    OWNER TO { new_owner | CURRENT_ROLE | CURRENT_USER |
SESSION_USER }
    REPLICA IDENTITY { DEFAULT | USING INDEX index_name | FULL |
NOTHING }

and table_constraint_using_index is:
```

```
    [ CONSTRAINT constraint_name ]
    { UNIQUE | PRIMARY KEY } USING INDEX index_name
    [ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED |
INITIALLY IMMEDIATE ]
```

```
-- System columns
--
-- https://www.postgresql.org/docs/current/ddl-system-columns.html
SELECT
  tableoid,
  cmin, cmax,
  xmin, xmax,
  ctid FROM ...;
```

EXCLUDE constraint
Reference: https://www.postgresql.org/docs/current/sql-createtable.html#SQL-CREATETABLE-EXCLUDE

EXCLUDE
Generalized **UNIQUE** constraint
Must use an **index** supporting the type.

e.g. x + y must be unique

index **btree** supports integer

```
mydb# \dAc
                                    List of operator classes
   AM   |         Input type          | Storage type |
Operator class        | Default?
========+=============================+==============+=================
================+=========
 brin   | "char"                      |              |
char_bloom_ops               | no
 brin   | "char"                      |              |
char_minmax_ops              | yes
 brin   | anyrange                    |              |
range_inclusion_ops          | yes
 brin   | bytea                       |              |
bytea_minmax_ops             | yes
 ...
 btree  | double precision            |              | float8_ops
| yes
 btree  | inet                        |              | cidr_ops
| no
 btree  | inet                        |              | inet_ops
| yes
 btree  | integer                     |              | int4_ops
```

```
| yes
 ...
```

Define constraint with **btree**.

```
CREATE TABLE test (
  x INTEGER,
  y INTEGER,
  EXCLUDE btree (x+y WITH =)
)

INSERT INTO test (x, y) VALUES (1, 2);     -- OK
INSERT INTO test (x, y) VALUES (2, 1);     -- ERROR since 1+2 = 2+1
```

DEFERABLE/IMMEDIATE constraint
SET CONSTRAINT: https://www.postgresql.org/docs/current/sql-set-constraints.html

```
SET CONSTRAINTS { ALL | name [, ...] } { DEFERRED | IMMEDIATE }
```

DEFERABLE/IMMEDIATE constraint
Only relevant when using a **transaction**.
A **DEFERRABLE** constraint may defer the constraint check until transaction **COMMIT**.
Constraints are **IMMEDIATE** by default.

IMMEDIATE	[default] check constraint on statement execution
DEFERRABLE [INITIALLY IMMEDIATE]	deferrable but **deferring is still disabled** at the start of a transaction Use **SET CONSTRAINTS** <constraint name> DEFERRED to enable deferring
DEFERRABLE INITIALLY DEFERRED	deferred and deferring is enable at the start of a transaction

e.g.
Table definition

```
CREATE TABLE IF NOT EXISTS mytable (
  value INT PRIMARY KEY DEFERRABLE                          --
DEFERRED INITIALLY IMMEDIATE
  message TEXT UNIQUE DEFERRABLE INITIALLY DEFERRED
);
```

case: no transaction
Constraints are all applied IMMEDIATEly

```
```

```
> INSERT INTO mytable VALUES (1, 'a'), (2, 'a');
ERROR:  duplicate key value violates unique constraint
"mytable_message_key"
DETAIL:  Key (message)=(a) already exists.
```

case: transaction
Constraints may be deferred

```
> BEGIN ;

> INSERT INTO mytable VALUES (1, 'a'), (2, 'a');
        -- OK     case: message check is deferred by default

> INSERT INTO mytable VALUES (1, 'a'), (1, 'a');
        -- ERROR  case: value check is NOT deferred by default
ERROR:  duplicate key value violates unique constraint
"mytable_pkey"
DETAIL:  Key (value)=(1) already exists.

> SET CONSTRAINTS mytable_pkey DEFERRED;
        -- OK     case: all checks are deferred by default

> INSERT INTO mytable VALUES (1, 'a'), (1, 'a');


> COMMIT ;
        -- constraints applied on commit
ERROR:  duplicate key value violates unique constraint
"mytable_message_key"
DETAIL:  Key (message)=(a) already exists.
```

Table and Composite (aka Row) type
Row type: https://www.postgresql.org/docs/current/rowtypes.html

Every table automatically has a **row type** of the **same name**.

e.g.
1) Create table

```
-- Create table
CREATE TABLE mytable (            -- CREATE TYPE mytable AS (id
INT, value TEXT)
  id INT,
  value TEXT
);
INSERT INTO mytable VALUES (1,'');
```

2) Use as **value**

```
-- create value
SELECT (1,'a')::mytable;               -- row
  row
═══════
 (1,'a')

SELECT ((1,'a')::mytable).*;           -- unwrapped row
 id | value
════╪═══════
  1 | a
```

Use in **column definition**

```
CREATE TABLE mytable2 (
  t mytable
);
INSERT INTO mytable2 VALUES (ROW(1,'a'));

SELECT * FROM mytable2;                -- row
   t
═══════
 (1,a)

SELECT (t).* FROM mytable2;            -- unwrapped row
 id | value
════╪═══════
  1 | a
```

3) Use in SELECT

```
SELECT mytable FROM mytable;               -- SELECT "mytable" with
type mytable
SELECT "mytable" FROM mytable;             -- same
SELECT mytable::mytable FROM mytable;      -- same explicit
 mytable
═══════
 (1,"")
(1 row)

-- with alias
SELECT t FROM mytable t;
SELECT t::mytable FROM mytable t;          -- same explicit
   t
═══════
 (1,"")
(1 row)

-- with unwrap
SELECT * FROM mytable;
SELECT mytable.* FROM mytable;             -- same explicit
```

```
 id | value
════╪═══════
  1 |
```

*** [Postgres >11] fast ALTER TABLE .. ADD COLUMN with DEFAULT**
Reference: https://brandur.org/postgres-default

Postgres 11 release notes: https://www.postgresql.org/docs/11/release-11.html

```
Allow ALTER TABLE to add a column with a non-null default without
doing a table rewrite (Andrew Dunstan, Serge Rielau)
```

ALTER TABLE command reference

```
When a column is added with ADD COLUMN and a non-volatile DEFAULT
is specified, the default value is evaluated at the time of the
statement and the result stored in the table's metadata, where it
will be returned when any existing rows are accessed. The value
will be only applied when the table is rewritten, making the ALTER
TABLE very fast even on large tables. If no column constraints are
specified, NULL is used as the DEFAULT. In neither case is a
rewrite of the table required.
```

How to add a NOT NULL column: https://stackoverflow.com/questions/512451/how-can-i-add-a-column-that-doesnt-allow-nulls-in-a-postgresql-database

*** INSERT**

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
INSERT INTO table_name [ AS alias ] [ ( column_name [, ...] ) ]
    [ OVERRIDING { SYSTEM | USER } VALUE ]
    { DEFAULT VALUES | VALUES ( { expression | DEFAULT } [, ...] )
[, ...] | query }
    [ ON CONFLICT [ conflict_target ] conflict_action ]
    [ RETURNING { * | output_expression [ [ AS ] output_name ] }
[, ...] ]

where conflict_target can be one of:

    ( { index_column_name | ( index_expression ) } [ COLLATE
collation ] [ opclass ] [, ...] ) [ WHERE index_predicate ]
    ON CONSTRAINT constraint_name

and conflict_action is one of:

    DO NOTHING
    DO UPDATE SET { column_name = { expression | DEFAULT } |
                    ( column_name [, ...] ) = [ ROW ]
```

```
( { expression | DEFAULT } [, ...] ) |
                    ( column_name [, ...] ) = ( sub-SELECT )
                  } [, ...]
              [ WHERE condition ]
```

ON CONFLICT
Allows using **UPDATE** instead of INSERT if the record already exists.

e.g.

```
INSERT INTO t1 (id, x)
  (SELECT uid, y FROM t2)
  ON CONFLICT (id)
   DO UPDATE SET x = EXCLUDED.y;

-- same with MERGE
MERGE INTO t1
  USING  (SELECT uid, y FROM t2) as t2
  WHEN MATCHED THEN
    UPDATE SET x = t2.y;
```

```
-- ON CONFLICT [Non-standard SQL]
--
-- Perform UPDATE instead of INSERT if the row already exists
INSERT INTO ...
  ON CONFLICT (id)            -- specify conflict by violated
column
  ON CONFLICT id_pkey         -- specify conflict by violated
constraint

  -- actions
  DO NOTHING                  -- ignore conflict
  DO UPDATE SET               -- use UPDATE instead of INSERT
    x = EXCLUDED.u,           -- use EXCLUDED
    y = EXCLUDED.y,           -- multiple
    ...
      WHERE ...
```

* DELETE
DELETE: https://www.postgresql.org/docs/current/sql-delete.html

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
DELETE FROM [ ONLY ] table_name [ * ] [ [ AS ] alias ]
    [ USING from_item [, ...] ]
    [ WHERE condition | WHERE CURRENT OF cursor_name ]
    [ RETURNING [ WITH ( { OLD | NEW } AS output_alias [, ...] ) ]
                { * | output_expression [ [ AS ] output_name ] }
```

```
[, ...] ]
```

==========

== Partitioning ==

Partitioning: https://www.postgresql.org/docs/current/ddl-partitioning.html

(Declarative) Partitioning
The original table is the **partitioned table**.
The partitioned table is a **virtual table** i.e. it has no storage.
The actual data is stored in child tables called **partitions**.

A partition is **attached** to the partitioned table.
A partition may be **detached** from the partitioned table.

Declarative vs inheritance based partition
Old way of partitioning was **inheritance** based
i.e. partitioned table as **parent**, and partitions as **child tables INHERITing** from parent.
The partitions were **implicit** as opposed to declarative
e.g. no **explicit declaration** of which data goes in which partition.

Declarative was introduced in **PostgreSQL v10**.
e.g. declare partition 100 ~ 200
ATTACH PARTITION t FOR VALUES FROM (100) TO (200)

Table definition inheritance
Under the hood, a partition **INHERITs** from the parent table.
Hence, similar rules apply:

	defined on child	set on ATTACH	can DROP
column name column data type	MUST	ERROR if different	NO
NOT NULL	MUST	ERROR if different	NO
CHECK	MUST	ERROR if different	NO
DEFAULT	optional	YES	YES
PRIMARY KEY	optional	YES	NO
UNIQUE KEY	optional	YES	NO
EXCLUDE	optional	YES	NO
GENERATED identity	NO	YES uses **SEQUENCE instance** as parent	NO
GENERATED expression	MUST	NO	NO

Indexes All indexes i.e. not index just for UNIQUE, PRIMARY KEY,	optional	YES	NO

3 types of **partition methods**:
- **LIST**: partition by explicit list of values

```
CREATE TEMPORARY TABLE test_list (
  id INT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
  value INT NOT NULL,
  PRIMARY KEY(id, value)
)
PARTITION BY LIST (value);

CREATE TEMPORARY TABLE test_list_1 PARTITION OF test_list (value)
                  -- partition for value in [0,1,2]
FOR VALUES IN (0, 1, 2);

CREATE TEMPORARY TABLE test_list_2 PARTITION OF test_list (value)
                  -- partition for value in [3, 4]
FOR VALUES IN (3, 4);

CREATE TEMPORARY TABLE test_list_rest PARTITION OF test_list
(value) DEFAULT;      -- partition for everything else
```

- **RANGE**: partition by range of values

- **HASH**: partition by numeric **modulus**

e.g.

```
CREATE TEMPORARY TABLE test_hash (
  id INT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
  value INT NOT NULL,
  PRIMARY KEY(id, value)
)
PARTITION BY HASH (value);

CREATE TEMPORARY TABLE test_hash_0 PARTITION OF test_hash (value)
         -- partition for value % 3 = 0
FOR VALUES WITH (MODULUS 3, REMAINDER 0);

CREATE TEMPORARY TABLE test_hash_1 PARTITION OF test_hash (value)
         -- partition for value % 3 = 1
FOR VALUES WITH (MODULUS 3, REMAINDER 1);

CREATE TEMPORARY TABLE test_hash_2 PARTITION OF test_hash (value)
         -- partition for value % 3 = 2
FOR VALUES WITH (MODULUS 3, REMAINDER 2);
```

NOTE: HASH partition cannot have a default

```
CREATE TEMPORARY TABLE test_hash_2 PARTITION OF test_hash (value)
         -- ERROR
DEFAULT;
```

Partition pruning
setting: **enable_partition_pruning**
One of the main **optimizations** of using partitions:
Uses the **partition key** to filter out **tables** to check, instead of scanning all partitions.
Similar to **index**, but does NOT require an index.

index key	filter subset of rows
partition key	filter partition table

e.g.

```
-- Table definition
CREATE TEMPORARY TABLE test_list (
  id INT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
  key INT,
  PRIMARY KEY(id, key)
)
PARTITION BY LIST (key);

-- Partitions
CREATE TEMPORARY TABLE test_list_1 PARTITION OF test_list (key)
FOR VALUES IN (0, 1, 2);

... other partitions ...


-- e.g. with pruning
>  explain select * from test_list where key = 1;
                               QUERY PLAN
_____________________________________________________________________________
_____
 Seq Scan on test_list_1 test_list  (cost=0.00..103.28 rows=31
width=8)              -- GOOD: only scans one partition
   Filter: (key = 1)


-- e.g. without pruning
> set enable_partition_pruning to off;
> explain select * from test_list where key = 1;
                              QUERY PLAN
_____________________________________________________________________________
_
 Append  (cost=0.00..172.38 rows=51 width=8)
   ->  Seq Scan on test_list_1  (cost=0.00..103.28 rows=31
width=8)                  -- BAD: scans all partitions
         Filter: (key = 1)
```

```
   ->  Seq Scan on test_list_2  (cost=0.00..68.85 rows=20 width=8)
         Filter: (key = 1)
```

Usage: basic

Step 1) Create **partitioned table** with **PARTITION BY**

```
CREATE TABLE test_partitioned (
    -- Various constraints inherited by partition
  id INT NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
    -- PRIMARY KEY, IDENTITY
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    -- DEFAULT
  value VARCHAR NOT NULL DEFAULT '',

  x INT NOT NULL DEFAULT 1 CHECK (x > 0),
    -- CHECK constraint
  x2 INT NOT NULL GENERATED ALWAYS AS (x * 2) STORED,
    -- GENERATED
  UNIQUE(id, value)
    -- UNIQUE constraint
)
PARTITION BY RANGE (id);

CREATE INDEX on test_partitioned (value);
    -- custom index
```

Step 2) Create partitions with **PARTITION OF**

```
CREATE TABLE test_partitioned_1 PARTITION OF test_partitioned
  FOR VALUES FROM (0) TO (1000);

CREATE TABLE test_partitioned_rest PARTITION OF test_partitioned
DEFAULT;
```

Partitions **inherit** columns and constraints

```
> \d test_partitioned

                              Table "pg_temp_0.test_partitioned"
 Column  |            Type             | Collation | Nullable |
Default
---------+-----------------------------+-----------+----------+----
--------------------------------
 id      | integer                     |           | not null |
generated by default as identity
 created | timestamp without time zone |           | not null |
```

```
CURRENT_TIMESTAMP
 value   | character varying           |           | not null |
''::character varying
 x       | integer                     |           | not null | 1
 x2      | integer                     |           | not null |
generated always as (x * 2) stored
Partition key: RANGE (id)
Indexes:
    "test_partitioned_pkey" PRIMARY KEY, btree (id)
    "test_partitioned_id_value_key" UNIQUE CONSTRAINT, btree (id,
value)
    "test_partitioned_value_idx" btree (value)
Check constraints:
    "test_partitioned_x_check" CHECK (x > 0)
Number of partitions: 2 (Use \d+ to list them.)
Partitions: test_partitioned_1 FOR VALUES FROM (0) TO (1000),
            test_partitioned_rest FOR VALUES FROM (1000) TO
(MAXVALUE)


> \d test_partitioned_rest
                            Table
"pg_temp_0.test_partitioned_rest"
 Column  |            Type             | Collation | Nullable |
Default
---------+-----------------------------+-----------+----------+----
---------------------------------
 id      | integer                     |           | not null |
generated by default as identity
 created | timestamp without time zone |           | not null |
CURRENT_TIMESTAMP
 value   | character varying           |           | not null |
''::character varying
 x       | integer                     |           | not null | 1
 x2      | integer                     |           | not null |
generated always as (x * 2) stored
Partition of: test_partitioned FOR VALUES FROM (2000) TO (MAXVALUE)
Indexes:
    "test_partitioned_rest_pkey" PRIMARY KEY, btree (id)
    "test_partitioned_rest_id_value_key" UNIQUE CONSTRAINT, btree
(id, value)
    "test_partitioned_rest_value_idx" btree (value)
Check constraints:
    "test_partitioned_x_check" CHECK (x > 0)
```

Step 3) Add data

```
INSERT INTO test_partitioned (value)
  (SELECT format('value%s', x * x) FROM generate_series(1, 10000)
AS t(x));
```

* Usage: attach existing table with **ATTACH PARTITION**

IMPORTANT: not required, but should have a **CHECK** constraint on the partition equivalent to the partition.
Otherwise, the table data will be **locked** and scanned to validate the partition.

e.g.

```
-- Partitioned table
CREATE TABLE test_partitioned (
  id INT NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  ...
) PARTITION BY RANGE(id);

CREATE TABLE test_partitioned_rest PARTITION OF test_partitioned
DEFAULT;
```

Attach:

```
-- IMPORTANT: CHECK on new partition
-- Without this, the whole new partition will be locked and
-- scanned to validate that its data does not violate the range.
ALTER TABLE test_partitioned_1 ADD CONSTRAINT temp_check
CHECK(:range);

-- IMPORTANT: CHECK on DEFAULT partition, if any
-- Without this, the whole default partition will be locked and
-- scanned to validate that its data does not violate the range.
ALTER TABLE test_partitioned_rest ADD CONSTRAINT temp_check_default
CHECK(NOT (:range));

ALTER TABLE test_partitioned ATTACH PARTITION test_partitioned_1
FOR VALUES FROM (1000) to (2000);

-- DROP the temporary checks.
ALTER TABLE test_partitioned_1 DROP CONSTRAINT temp_check;
ALTER TABLE test_partitioned_rest DROP CONSTRAINT
temp_check_default;
```

Usage: create partition with **CREATE TABLE .. LIKE**
Ref: https://www.postgresql.org/docs/current/ddl-partitioning.html

e.g.

Step 1) Create base table:

```
-- Create initial table
CREATE TABLE test_partitioned (
```

```
  id INT NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
         -- PRIMARY KEY, IDENTITY column
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
         -- DEFAULT column
  value VARCHAR NOT NULL DEFAULT '',
  x INT NOT NULL DEFAULT 1 CHECK (x > 0),
         -- CHECK column
  x2 INT NOT NULL GENERATED ALWAYS AS (x * 2) STORED,
         -- GENERATED expression column
  UNIQUE(id, value)
         -- UNIQUE
)
PARTITION BY RANGE (id);


CREATE INDEX on test_partitioned (value);
         -- additional index


-- Create a partition
CREATE TABLE test_partitioned_rest PARTITION OF test_partitioned
DEFAULT;

-- Populate
INSERT INTO test_partitioned (value)
  (SELECT format('value%s', x * x) FROM generate_series(0, 10000)
AS t(x));
```

Step 2) Create a partition with **LIKE**.

```
--
CREATE TABLE test_partitioned_1 (
  LIKE test_partitioned
  INCLUDING GENERATED                 -- MUST define GENERATED
expressions
  INCLUDING CONSTRAINTS               -- MUST define CHECK

  -- INCLUDING IDENTITY               -- NO. Cannot attach
  -- INCLUDING DEFAULTS               -- optional. NOT defined on
ATTACH
  -- INCLUDING INDEX                  -- optional. Will be defined
on ATTACH
);

-- Initial table
\d test_partitioned_1

                                Table "test_partitioned_1"
 Column  |            Type             | Collation | Nullable |
Default
```

```
--------+-----------------------------+-----------+----------+----
-------------------------------
 id      | integer                     |           | not null |
 created | timestamp without time zone |           | not null |
 value   | character varying           |           | not null |
 x       | integer                     |           | not null |
 x2      | integer                     |           | not null |
generated always as (x * 2) stored
Check constraints:
    "test_partitioned_x_check" CHECK (x > 0)
```

Add data

```
\set range 'id >= 1000 AND id < 2000'

-- e.g.
INSERT INTO test_partitioned_1 (SELECT :columns FROM
test_partitioned WHERE :range);
DELETE FROM test_partitioned_rest WHERE :range;
```

Step 3) Attach partition with **ATTACH PARTITION**

```
-- See ATTACH PARTITION usage above about the CHECKs
ALTER TABLE test_partitioned_1 ADD CONSTRAINT temp_check
CHECK(:range);
ALTER TABLE test_partitioned_rest ADD CONSTRAINT temp_check_default
CHECK(NOT (:range));

ALTER TABLE test_partitioned ATTACH PARTITION test_partitioned_1
FOR VALUES FROM (1000) to (2000);

ALTER TABLE test_partitioned_1 DROP CONSTRAINT temp_check;
ALTER TABLE test_partitioned_rest DROP CONSTRAINT
temp_check_default;
```

Table definition after ATTACH.

```
\d test_partitioned_1
\prompt 'Attached table' _
                              Table "test_partitioned_1"
 Column  |            Type             | Collation | Nullable |
Default
--------+-----------------------------+-----------+----------+----
-------------------------------
 id      | integer                     |           | not null |
generated by default as identity      -- IDENTITY added
 created | timestamp without time zone |           | not null |
 value   | character varying           |           | not null |
 x       | integer                     |           | not null |
 x2      | integer                     |           | not null |
```

```
generated always as (x * 2) stored
Partition of: test_partitioned FOR VALUES FROM (1000) TO (2000)
Indexes:
    "test_partitioned_1_pkey" PRIMARY KEY, btree (id)
                                       -- PRIMARY KEY added
    "test_partitioned_1_id_value_key" UNIQUE CONSTRAINT, btree (id,
value)                              -- UNIQUE added
    "test_partitioned_1_value_idx" btree (value)
                                       -- index added
Check constraints:
    "test_partitioned_x_check" CHECK (x > 0)
```

Usage: **DEFAULT** partition

DEFAULT partition
By default **INSERT fails** if there is no partition to put a given data.
Only for **LIST** and **RANGE** partitions.

e.g.
Step 1) Create base table

```
CREATE TABLE test_partitioned (
  id INT NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
)
PARTITION BY RANGE (id);

INSERT INTO test_partitioned (id) VALUES (1), (1000), (3000);
    -- ERROR. No partition to hold the data
```

Step 2) Create **DEFAULT partition** and add dataa

```
CREATE TABLE test_partitioned_1
   PARTITION OF test_partitioned
   FOR VALUES FROM (1000) TO (3000);

CREATE TABLE test_partitioned_rest
   PARTITION OF test_partitioned DEFAULT;

INSERT INTO test_partitioned (id) VALUES
   (1),                                        --
test_partitioned_rest
   (1000),                                     --
test_partitioned_1
   (3000);                                     --
test_partitioned_rest
```

DEFAULT vs **MAXVALUE/MINVALUE**
DEFAULT catches **all data** that does not have an explicit other partition.

MINVALUE/MAXVALUE only catches within a range.

e.g.

```
CREATE TABLE test_partitioned (
  id INT NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
)
PARTITION BY RANGE (id);

CREATE TABLE test_partitioned_1
   PARTITION OF test_partitioned FOR VALUES FROM (1000) TO
(MAXVALUE);

INSERT INT test_partitioned (id) VALUES (1000);         // ERROR
INSERT INT test_partitioned (id) VALUES (1);            // OK
```

*** GOTCHA: IDENTITY column inheritance**

All **IDENTITY** columns are **inherited** by child tables, even if it's not the partition key. Hence, child columns can NOT have

e.g. parent table

```
CREATE TABLE test_partitioned (
  id INT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
  id2 INT NOT NULL GENERATED BY DEFAULT AS IDENTITY
) PARTITION BY RANGE (id);
```

GOOD case: create child without IDENTITY

```
CREATE TABLE child_1 (
  LIKE parent
  INCLUDING ALL
  EXCLUDING IDENTITY          -- GOOD. Should exclude all identity
columns
);
ALTER TABLE parent ATTACH PARTITION child_1 FOR VALUES FROM (0) to
(100);

-- NOTE: IDENTITY is inherited from
\d child_1

 Column  |            Type             | Collation | Nullable |
Default
---------+-----------------------------+-----------+----------+----
-------------------------------
 id      | integer                     |           | not null |
generated by default as identity
 id2     | integer                     |           | not null |
generated by default as identity
```

BAD case: child with IDENTITY

```
CREATE TABLE child_2 (
  LIKE parent
  INCLUDING ALL
);

-- BAD: partition key id is IDENTITY
ALTER TABLE parent ATTACH PARTITION child_2 FOR VALUES FROM (100)
to (200);

 table "child_2" being attached contains an identity column "id"

ALTER TABLE child_2 ALTER COLUMN id DROP IDENTITY;


-- BAD: non-partition key id2 is IDENTITY
ALTER TABLE parent ATTACH PARTITION child_2 FOR VALUES FROM (100)
to (200);

 table "child_2" being attached contains an identity column "id2"

ALTER TABLE child_2 ALTER COLUMN id DROP IDENTITY;
ALTER TABLE parent ATTACH PARTITION child_2 FOR VALUES FROM (100)
to (200);
```

==========

==View ==

Views/materialized views
Views: https://www.postgresql.org/docs/current/rules-views.html
Materialized view: https://www.postgresql.org/docs/current/rules-materializedviews.html

View
Materialized view
A view is effectively a **virtual table**
i.e. a table generated from a SELECT * statement

Materialized vs normal view
Normal views .
Materialized views need to be **updated i.e. refreshed** explicitly.

```
-- View
CREATE VIEW myview AS
  SELECT ...


-- Materialized
CREATE MATERIALIZED VIEW myview AS
   SELECT ...
```

```
REFRESH MATERIALIZED VIEW myview;
```

==========